Food Before It Spoils: Deadline-Aware Food Delivery Routing

CS201 Project Report · Beyond the Classic TSP

Group [G1-5]: Looi Le, Thu Thu, Hanlin, Rae, Geng Hong

Summary

One rider collects a batch of orders at a restaurant and delivers them in one trip. Each dish stays fresh for a different time (ice cream 30 minutes, noodles 60, rice 90), so the customer who ordered first is not always the one whose food spoils first. The classic Travelling Salesperson Problem (TSP) treats every city as equally important, ignores *when* each city is reached, and must visit every city. None of that holds here.

We define a deadline-aware variant of the TSP with an open route, optional (cancellable) customers and two ranked goals: **(1) deliver as many orders on time as possible, then (2) minimise the average wait of the delivered customers**. We redesign the algorithm and its data structures in two meaningfully different ways:

- **Approach A, plan-and-fix:** keep a *complete plan* (a permutation of all customers), start from deadline order, and repeatedly apply the best "move one stop" improvement.
- **Approach B, greedy with lookahead:** build the route *one stop at a time*, choosing the next stop by a strict ranking of five rules, the first of which counts how many other orders that stop would doom.

Both are compared with a nearest-neighbour baseline and an exact solver on **540 inputs built from the GeoNames `MY.txt` dataset** (real populated places around Bukit Bintang, Kuala Lumpur). For realistic batch sizes (5 to 20 orders) Approach A cancels the fewest orders and matches the exact optimum on deliveries in 83 to 100% of cases. With 50 or more orders the ranking flips: Approach B and even nearest neighbour keep more orders, and run about 1,000 times faster.

1. Motivation and use case

1.1 The real-world scenario

Food delivery platforms batch several orders from one restaurant onto one rider. The batch is only useful if each dish arrives while it is still good to eat. Melted ice cream or cold noodles lead to complaints, refunds and lost customers, so a late delivery is usually worse than an honest cancellation that is refunded immediately.

Dishes in the same batch have very different shelf lives, and the customers are spread over a city, so the rider faces a real conflict: the nearest customer may have the most robust food, and the most urgent customer may be the farthest away.

A small example from our slides makes the conflict concrete. Five customers live on one road with the restaurant in the middle. The nearest customer, Dan, ordered rice (90 minutes); the two ice creams (30 minutes) are the farthest east. Driving to the closest customer every time (nearest neighbour) reaches both ice creams after 30 minutes, so only 3 of 5 orders arrive fresh. A deadline-aware plan delivers all 5.

1.2 Why the classic TSP does not address it

Classic TSP assumption	What our use case needs instead
Every city is equally important	Each order has its own deadline, set by its food's shelf life
Only the visit order matters	The **arrival time** at each stop matters: it decides freshness and waiting
One goal: shortest total distance	Two ranked goals: most orders on time, then lowest average wait
Every city must be visited	Orders that cannot arrive on time are **cancelled** (left out of the route)
The tour returns to the start	The route is **open**: it ends at the last customer

The shortest tour can be a poor delivery route. Total distance treats the first and the last stop the same, but every minute spent early in the route delays *every* later customer. A distance-optimal tour may also leave the most urgent dish until the end. Because the objective changes from "total length" to "how many deadlines are met, then the sum of arrival times", the classic algorithms (and nearest neighbour, which only looks at distance) no longer target the right goal, and the redesign below is needed.

2. The new TSP variant

2.1 Input

All information is known when the rider leaves; no new orders arrive later.

- One restaurant r and n customers.
- Travel time w(i, j) in minutes between any two places (symmetric).
- For each customer k: order time o_k (minutes, ≤ 0), food type and shelf life ℓ_k.
- The rider leaves the restaurant at t = 0, the time of the last order.

2.2 Output

A sequence of distinct customers $\pi = (\pi_1, \ldots, \pi_m)$, $m \le n$, giving the order in which the rider visits them, starting at *r*. Customers not in π are **cancelled**.

2.3 Constraints

- One rider, who carries all orders and does not return to the restaurant.
- Arrival times: $a(\pi_1) = w(r, \pi_1)$ and $a(\pi_{i+1}) = a(\pi_i) + w(\pi_i, \pi_{i+1})$ (no service time at the door).
- Deadline $d_k = o_k + \ell_k$. Every delivered order must satisfy $a_k \le d_k$ (late = 0). Arriving exactly on the deadline counts as on time.
- Wait of a delivered customer: $wait_k = a_k - o_k$. Since $a_k \le d_k$ is the same as $wait_k \le \ell_k$, the deadline is simply a wait limit.

2.4 Objectives, in strict order

1. Maximise the number of delivered orders, *m* (equivalently, minimise cancellations).
2. Among routes with the same *m*, minimise the average wait $(1/m) \cdot \Sigma$ over delivered k of $(a_k - o_k)$.

We also report the longest wait and the number of cancelled orders. The goals are compared **lexicographically, with no weights**. This matters: cancelling a far-away customer always makes the average wait look better, so ranking by wait alone would reward cancelling customers. The worked example in Section 8.5 shows nearest neighbour doing exactly that.

For a fixed delivered set, minimising average wait is the same as minimising the sum of arrival times. With different delivered sets the order times differ, so our scorer always uses $a_k - o_k$ directly.

2.5 Relation to known problems

With loose deadlines and every customer delivered, the problem is the open-path **Minimum Latency (Travelling Repairman) Problem**, which is NP-hard. Deadlines and optional customers make it a deadline-constrained, orienteering-style generalisation. We therefore do not aim for optimal solutions on large inputs; we design heuristics and measure them against an exact solver where one can finish.

3. Assumptions

The team made the following assumptions. They are set in `src/com/delivery/tools/Common.java` and the problem definition, are shared by every method, and are documented in `docs/data_decisions.md` and the approach write-ups.

3.1 Problem and model assumptions

#	Assumption	Reason / effect
A1	**Static batch.** All orders are known at departure; no orders arrive mid-route	Keeps the comparison fair and reproducible; dynamic arrivals are future work
A2	**One rider, one restaurant, one trip.** The rider carries every order and never returns to the restaurant	Matches a single batched pickup; no capacity limit is modelled
A3	The rider leaves at t = 0, the time of the **last** order. Earlier orders have negative order times	Waiting starts when the customer orders, not when the rider leaves
A4	**No cooking or preparation time**; all food is ready at departure	Isolates the routing decision
A5	**No service time** at each door	Arrival at the next stop depends only on travel
A6	**Symmetric travel, no traffic.** w(i,j) = w(j,i), constant over time	Time-dependent travel is outside scope
A7	Travel time = **straight-line (haversine) distance ÷ 20 km/h**	The low speed stands in for detours, traffic lights and jams. Straight-line times obey the triangle inequality, which Approach B's "doomed" test and the exact solver's pruning rely on
A8	**Deadline = order time + shelf life**, with shelf lives ice cream 30, noodles 60, rice 90 minutes	Illustrative values for comparing deadline pressure, not measured food-safety limits
A9	**Late food is never delivered.** An order that cannot arrive on time is cancelled	A route is valid only if late = 0; every returned route is re-checked
A10	Arriving **exactly on** the deadline is on time (tolerance 10^{-9})	Avoids floating-point false negatives
A11	**Cancelled first, wait second.** Goals are compared lexicographically, no weighting	Prevents "improving" the average by dropping customers
A12	Wait is measured over **delivered** customers only; an empty route has average wait 0	Cancelled customers have no arrival time
A13	Customers at the same coordinates are 0 minutes apart	Occurs only when locations are reused for n > 33

3.2 Data assumptions

#	Assumption	Reason / effect
D1	**Bukit Bintang Plaza** (GeoNameID 7284944, class S / MALL) stands in for the restaurant	`MY.txt` has no rows with feature code `REST`; Bukit Bintang is a busy food area in central KL
D2	Customers are **populated places (feature class P) within 6 km** of the restaurant	A compact urban delivery area with varied distances
D3	Each populated place stands for **an address in that area**, even when the point is not a real home (city centre, national palace, a park)	The solver uses only coordinates; removing points would reshuffle every test case
D4	Hotels (class S / HTL) are **not** used as customers	They cluster near the restaurant and would make the problem much easier (see Section 4.2)
D5	Rows with **identical coordinates** are duplicates; one is kept	7 rows dropped; the kept row is a `PPL` in every case
D6	Order times are uniform random in the **10 minutes before departure**, shifted so the latest order is at t = 0	All orders are already placed when the rider leaves
D7	Each order's food is ice cream, noodles or rice with **equal probability**	Mixed shelf lives create the urgency-versus-distance conflict
D8	When n > 33, **locations are reused** in shuffled rounds	Large *n* is used mainly for speed and overload, not route quality
D9	The **population** column is not used	Many of the 33 locations show 0, and the problem does not need it

3.3 Experimental assumptions

#	Assumption
E1	All methods read the **same 540 JSON inputs** and are scored by the **same evaluator** (`Evaluate.java`)
E2	Randomness comes from seeded `java.util.Random`, so every input is reproducible from (n, seed)
E3	The three shelf-life multipliers for one (n, seed) share the same locations, order times and foods; only shelf lives change. So the 540 inputs are 180 random scenarios × 3 tightness levels
E4	The exact solver is run only for n ≤ 10
E5	Run time is the median wall time of repeated runs on one machine; absolute times differ by machine, so only the relative sizes are meaningful

4. Dataset

4.1 Records and columns used

The primary source is the supplied GeoNames Malaysia file `data/raw/MY.txt` (71,709 rows, 19 columns, all well formed). It is kept unchanged. We use five columns: **GeoNameID, name, latitude, longitude and feature class**.

Step	Result
Start from `MY.txt`	71,709 rows
Restaurant	Bukit Bintang Plaza (GeoNameID 7284944)
Keep feature class P within 6 km of the restaurant (haversine)	40 places
Drop rows with exactly the same coordinates	7 dropped, **33 customer locations**
Per input: pick *n* locations and add order time, food and shelf life	One JSON instance

The 33 locations are listed in `data/processed/MY_33_locations.txt`; the seven dropped duplicates are listed in `docs/data_decisions.md`.

4.2 Why this subset

- **Realistic delivery radius.** 6 km around a central mall is a typical food-delivery catchment. At 20 km/h the farthest customer is about 18 minutes away, which is comparable to the 30-minute ice-cream shelf life, so deadlines genuinely conflict without making the problem hopeless.
- **Size fits the experiment.** Batches of 5 to 10 orders can be solved exactly for a ground truth, 20 is a large realistic batch, and 50 to 100 test speed and overload. 33 unique locations cover every size up to 20 without repetition.
- **Why not hotels.** Adding hotels would give 363 unique locations, but their median distance from the restaurant is 1.34 km (against 3.42 km for populated places), and 160 of them are within 1 km. Trips would become very short and few deadlines would conflict, which removes the interesting part of the problem. Changing the pool would also reshuffle every shared input.

4.3 Supplementary and derived attributes

These are generated by `GenerateInstance.java`, stored in the JSON inputs, and linked to `MY.txt` by GeoNameID:

Attribute	How it is produced
Order time o_k	Uniform random within the 10 minutes before departure; latest order shifted to 0

Attribute	How it is produced
Food type	Ice cream, noodles or rice, chosen at random
Shelf life ℓ_k	30 / 60 / 90 minutes × the tightness multiplier (0.5, 1.0 or 1.5)
Travel-time matrix	(n+1) × (n+1), index 0 = restaurant, index k = customer k; haversine km ÷ 20 km/h × 60

Tightness	Ice cream	Noodles	Rice	Meaning
0.5	15	30	45	Strict deadlines, heavy conflict
1.0	30	60	90	Base case
1.5	45	90	135	Relaxed deadlines

4.4 Known data limitations

- There are no restaurants in `MY.txt`, so a mall stands in for the restaurant.
- Some coordinates are approximate: five villages share one exact point.
- Some points are not real customer homes (Kuala Lumpur city centre, Istana Negara, Lake Gardens); Bukit Bintang deais only 0.1 km from the restaurant.
- Above 33 orders locations repeat, so n = 50 and 100 measure speed and overload rather than route quality.

5. Shared data structures and reference methods

5.1 Data model

Class	Holds
`Customer`	Record `(id, orderTime, shelfLife, foodType)` with `deadline()`
`DeliveryInstance`	The travel matrix (adjacency matrix of a complete graph) and the customer list. Index 0 is the restaurant, index `i + 1` is customer `i`; the constructor rejects a matrix of the wrong size
`RouteResult`	Output: `route`, `arrivals`, `dropped` (cancelled) and `meanWait`

The travel matrix stays a dense adjacency matrix, as in the classic TSP: every pair is a possible move and lookups are O(1). What changes is what each vertex carries (a deadline and an order time) and what the algorithms track while building a route (current time, feasibility and the plan).

5.2 Baseline: nearest neighbour (`NearestNeighbor.java`)

Classic nearest neighbour from the brief, adapted to an open route: always drive to the closest unvisited customer, then drop every order that arrives late. It ignores deadlines while routing. Time $O(n^2)$.

5.3 Exact reference (`BruteForce.java`)

A depth-first search over on-time routes for n ≤ 10. Only on-time stops are extended, so every leaf is valid. Two prunings keep it exact and fast:

- **Bound:** delivered so far + orders still *directly* reachable cannot beat the best → stop. An order not directly reachable now can never be reached later (triangle inequality).
- **Dominance:** for the same (visited set, last customer), a state that is no earlier and has no smaller wait sum than one already seen cannot lead to anything better → stop.

Worst case O(n! · n). A plain version without dominance is kept for checking the pruned one.

5.4 Scorer (`Evaluate.java`)

Re-drives any returned route from the instance file, recomputes arrivals, and reports delivered, cancelled, **late**, average wait and longest wait in O(n). The experiment runner rejects any route with late > 0, so no method can report an invalid result.

6. Approach A: deadline order, then improve by moving stops

6.1 Idea

Approach A is "sort, then fix". It always holds a **complete plan**: an ordering of *all* customers, including those that will be cancelled. To score a plan, the rider *walks* it: for each customer in plan order, drive there and deliver if it can be reached by its deadline, otherwise cancel it and stay put. Every walk is therefore a valid route.

1. **Start plan:** sort customers by (deadline, id), most urgent first (Earliest Deadline First).
2. **Improve:** try every move that takes one customer out and reinserts it at another position. Keep the best move if it is strictly better (more delivered, then lower average wait). Repeat until no move helps.

6.2 Redesigned data structure

Structure	Type	Role
`plan`	`List<Integer>`, size *n*	A permutation of all customer ids: the order in which the rider *tries* customers
`best`	`RouteResult`	Result of walking the current plan

The key design choice is to **keep cancelled customers inside the plan**. In a classic greedy, once an order is out of reach it is gone. In A, a later move can bring it earlier in the plan, where it becomes deliverable again. This turns the problem of choosing

a subset *and* an order into a search over permutations only, with the walk deciding the subset.

Alternatives considered: a pure Earliest-Deadline-First priority queue (pop most urgent, deliver or cancel) ignores the map; we use EDF only as the start plan, and a single sort is enough, so no heap is needed. A "least slack next" greedy ignores lookahead. Swapping two stops is an alternative neighbourhood; "move one stop" was implemented. We did not run comparative trials of these alternatives.

6.3 Pseudocode

```
SolverA(instance):
    plan <- customer ids sorted by (deadline, id)    # O(n log n)
    best <- walk(plan)
    loop:
        bestPlan <- none
        for from in 0 .. n-1:
            for to in 0 .. n-1, to != from:
                cand <- plan with plan[from] moved to `to`  # O(n)
                res  <- walk(cand)                          # O(n)
                # better = more delivered, else lower mean wait
                if isBetter(res, best):
                    best <- res; bestPlan <- cand
        if bestPlan is none: break
        plan <- bestPlan
    return best

walk(plan):
    t <- 0; cur <- RESTAURANT
    for id in plan:
        arr <- t + travel(cur, id)
        if arr <= deadline(id): deliver id at arr; t <- arr; cur <- id
        else:                   cancel id
    return (route, arrivals, cancelled, mean wait)
```

Termination and quality: every accepted move is strictly better and there are finitely many plans, so A always stops. The final plan is never worse than plain deadline order.

6.4 Complexity

- Start plan: $O(n \log n)$.
- One improvement pass: $n(n-1)$ moves, each $O(n)$ to build and $O(n)$ to walk, so $O(n^3)$.
- Total: $O(P \cdot n^3)$, where P is the number of passes. P depends on the instance and has no small fixed bound; the runner does not record it, so we make no empirical claim about its growth.
- Space: $O(n)$ for the plan, a candidate and the best, on top of the $O(n^2)$ matrix.

6.5 Strengths and limitations

Strength	Limitation
Can undo any earlier choice and bring a cancelled order back	Local search: stops when no *single* move helps
Never worse than deadline order; reduces to "shorten the waits" when deadlines are loose	Slower: every candidate is walked from the start
Uses exactly the project's scoring rule, so "better" means the project's goals	Number of passes, and so run time, is hard to predict
Deterministic, no tuning weights	When most orders must be dropped, EDF sends the rider to far urgent orders first and single moves cannot rebuild the route

Known trap (Example 5 in `DemoA`): customers S (−5, d 16), T (+5, d 31), U (−12, d 24), V (+4, d 20) on a line. The best route V → T → S → U delivers all 4. Deadline order S, V, U, T loses U, and reaching the best route needs *two* stops moved ahead of S at once. Every single move still loses someone, so A stops at 3 of 4.

7. Approach B: greedy with one-step lookahead

7.1 Idea

Approach B builds the route **one stop at a time** and never revisits a choice. At each stop it asks: *"If I go to this customer next, how many other orders do I doom?"*

7.2 Selection rules (multi-criteria, lexicographic)

Every candidate *u* is ranked by these rules in strict order; a later rule only matters when all earlier ones tie. There are no weights.

#	Rule	Computed as	Serves
1	Feasibility filter	skip *u* unless $t + w(cur, u) \le d_u$	Never deliver late food
2	Fewest lost	count alive $v \neq u$ with $a_u + w(u, v) > d_v$	Goal 1: on-time count
3	Shortest travel	$w(cur, u)$	Goal 2: low wait
4	Least slack	$d_u - a_u$, most urgent first	Keeps tight orders safe
5	Lowest id	*u*	Deterministic output

Why rule 2 is sound: going from *u* straight to *v* is the fastest way to reach *v* after *u* (triangle inequality). If even that is late, *v* is lost on every route starting with *u*. The

test is one-sided: passing it does not guarantee *v* survives, because each *v* is checked on its own.

7.3 Redesigned data structures

Variable	Type	Meaning
`visited[v]`	`boolean[n]`	Customer *v* already delivered
`alive[v]`	`boolean[n]`	*v* is unvisited and still reachable on time if the rider drives straight there now
`currentNode, t`	`int, double`	Rider position and current time
`route, arrivals`	`List`	Delivered stops so far

Why not a priority queue? Every score (lost count, travel, slack) depends on where the rider is now, so *every* key changes after every move. Recomputing lost counts already costs $O(n^2)$ per step; a heap could not reuse old keys, and a linear scan finds the single winner just as fast.

7.4 Pseudocode

```
SolverB(instance):
    visited <- all false; cur <- RESTAURANT; t <- 0
    route, arrivals <- []
    loop:
        # O(n): unvisited orders still reachable on time
        alive <- { v unvisited : t + travel(cur,v) <= deadline(v) }
        best <- none
        for u in alive:                                    # O(n)
            arr   <- t + travel(cur, u)
            # O(n): orders doomed if the rider goes to u next
            lost  <- |{ v in alive, v != u :
                        arr + travel(u,v) > deadline(v) }|
            slack <- deadline(u) - arr
            # compare lexicographically
            if (lost, travel(cur,u), slack, u) < key(best):
                best <- u
        if best is none: break
        t <- t + travel(cur, best); visited[best] <- true
        append best to route, t to arrivals; cur <- best
    dropped <- all unvisited customers
    return (route, arrivals, dropped, mean wait)
```

7.5 Complexity

At most *n* steps × O(n) candidates × O(n) lost-count scan = **$O(n^3)$ time, fixed** (no passes). Space O(n) on top of the matrix.

7.6 Strengths and limitations

Strength	Limitation

Strength	Limitation
Never makes a move that a direct check will doom someone, when a safer move exists	Checks each remaining order on its own, so it cannot see the result if we went another route
Identical to nearest neighbour when deadlines are loose: a strict extension of the baseline	Greedy: never backtracks
Very fast, predictable run time	Does not directly minimise total wait when choosing a stop
Deterministic, simple arrays, no weights	

Known trap (Example 2 in `DemoB`): P (+5, d 10), Q (−5, d 16), R (−1, rice, d 90). B goes to R first (nearest, "0 lost" because P and Q each look reachable separately), then can save only one of P and Q: 2 of 3. Approach A's deadline order handles this case and delivers all 3 (P → R → Q).

8. Experimental comparison

8.1 How the two approaches differ

	Approach A	Approach B
Algorithm idea	Build a full plan, then local search	Constructive greedy, one stop at a time
Main data structure	Plan: permutation of all *n* customers	`visited[]` / `alive[]` arrays with linear scan
Selection logic	One sort, one move type, one comparison	Five ranked rules with a one-step lookahead
Can undo early choices	Yes	No
Time	$O(n^3)$ per pass × passes	$O(n^3)$, fixed
Typical trap	Fix needs two moves at once	Orders checked one at a time

They differ in both algorithm design and main data structure, not just in parameters.

8.2 Setup

- **Inputs:** 6 order counts (5, 8, 10, 20, 50, 100) × 3 tightness levels (0.5, 1.0, 1.5) × 30 seeds = **540 inputs**, in `data/processed/`.
- **Methods:** nearest neighbour (`NN`), Approach A (`A_plan_fix`), Approach B (`B_greedy`) and, for n ≤ 10, the exact solver (`BRUTE`): 1,890 runs in total.
- **Metrics:** delivered, cancelled, average wait, longest wait, run time, and for n ≤ 10 the gap to the exact best (extra cancellations, share of runs matching the best delivered count, share matching on both goals).
- **Validity:** every route is re-scored by `Evaluate.java`; all 1,890 have late = 0.

- **Correctness checks:** `run_tests.sh` / `run_tests.bat` run the hand-worked examples for A (5) and B (3), the template worked example and the 5-customer slide example including all 120 permutations. They end with `ALL TEST SUITES PASSED.` (re-run for this report).

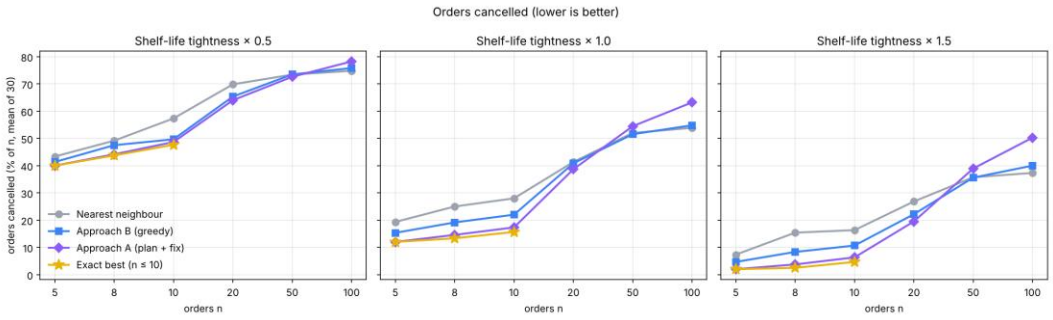
8.3 Results: orders cancelled (mean over 30 inputs)

Tightness 1.0 (base case):

n	Nearest neighbour	Approach B	Approach A	Exact best
5	0.97	0.77	**0.60**	0.60
8	2.00	1.53	**1.17**	1.07
10	2.80	2.20	**1.73**	1.57
20	8.27	8.17	**7.73**	–
50	26.00	**25.77**	27.23	–
100	**53.87**	54.80	63.20	–

All tightness levels (selected sizes):

Shelf life	n	NN	B	A	Exact
×0.5	10	5.73	4.97	**4.87**	4.77
×0.5	20	13.97	13.07	**12.80**	–
×0.5	100	**74.80**	75.80	78.23	–
×1.0	10	2.80	2.20	**1.73**	1.57
×1.0	20	8.27	8.17	**7.73**	–
×1.0	100	**53.87**	54.80	63.20	–
×1.5	10	1.63	1.07	**0.63**	0.47
×1.5	20	5.37	4.43	**3.90**	–
×1.5	100	**37.33**	40.00	50.27	–

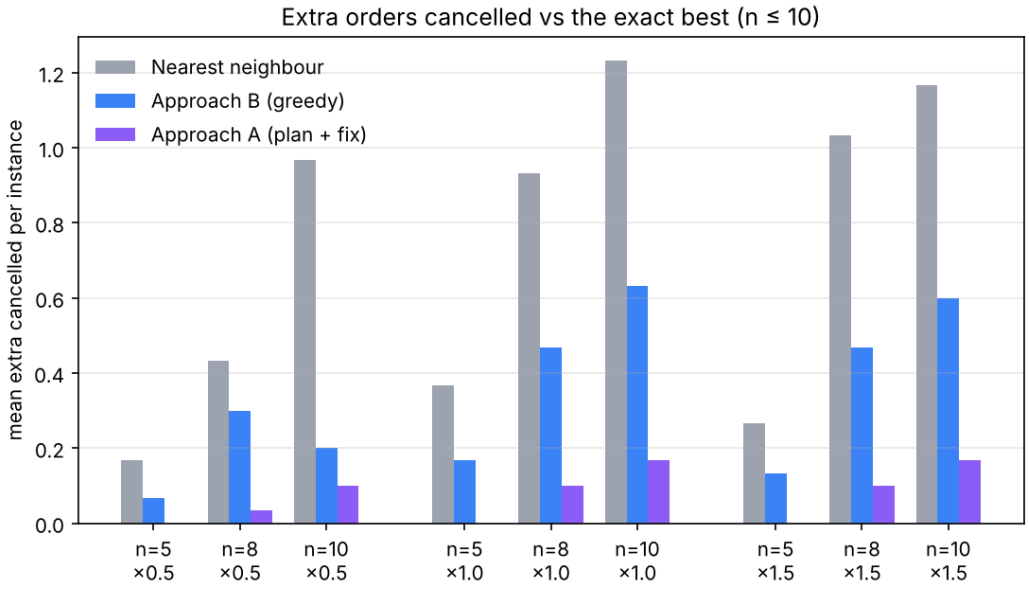


Orders cancelled (% of n) by input size and tightness

8.4 Results: gap to the exact best (n ≤ 10)

n	Tight	A: as many delivered as best	A: exactly best	B: as many delivered as best	B: exactly best	NN: as many delivered as best
5	1.0	100%	96.7%	83.3%	53.3%	66.7%
8	1.0	90.0%	66.7%	53.3%	33.3%	30.0%
10	1.0	83.3%	53.3%	46.7%	16.7%	26.7%
10	0.5	90.0%	63.3%	80.0%	33.3%	30.0%
10	1.5	83.3%	43.3%	46.7%	13.3%	30.0%

On average Approach A cancels at most 0.17 more orders than the optimum in any small setting; Approach B up to 0.63 more, and nearest neighbour up to 1.23 more.



Extra orders cancelled compared with the exact best, n ≤ 10

8.5 Worked example (template instance, n = 6, seed 10)

Solution	Route	Cancelled	Average wait	Longest wait
Exact best	1 → 2 → 5 → 4 → 6 → 3	0	29.0	62.7 (Sentul, rice)
Nearest neighbour	1 → 2 → 4 → 6 → 3	1 (Kuala Lumpur Village, ice cream)	21.9	49.8

The exact route serves the three ice creams first and leaves the rice for last. Nearest neighbour's average wait looks lower only because it served one fewer customer, which is why the goals are compared in order.

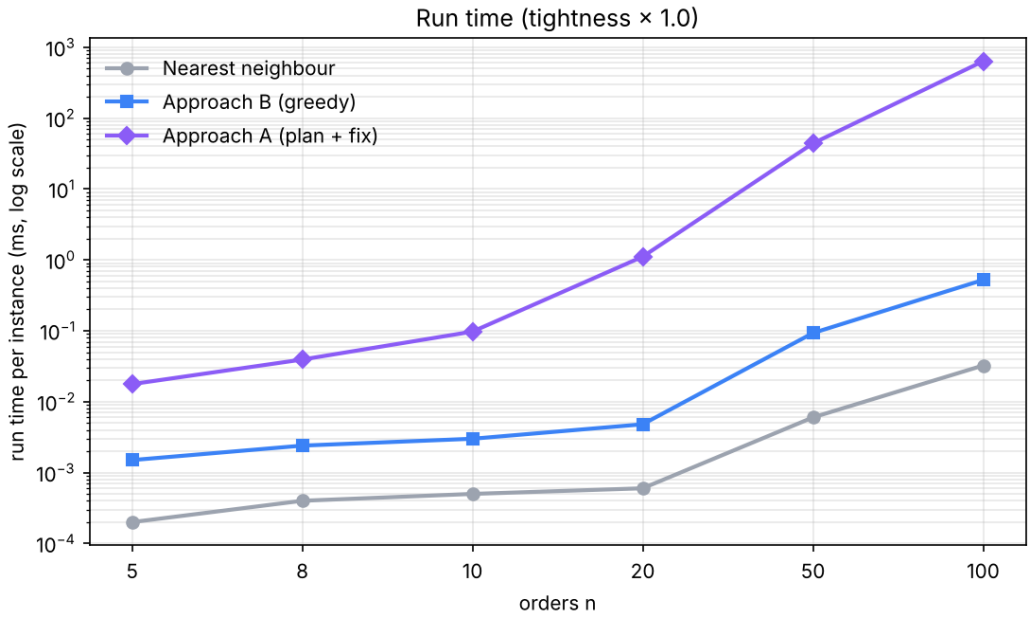
8.6 Head-to-head: A against B on the same input

Counting, per input, which approach wins in goal order (fewer cancelled, then lower average wait), over the 90 inputs per size (3 tightness × 30 seeds), computed from `results/raw_results.csv`:

n	A better	B better	Tie
5	32	1	57
8	49	18	23
10	56	19	15
20	53	35	2
50	27	62	1
100	9	81	0

8.7 Run time

Median wall time per input at n = 100, tightness 1.0: **Approach A ≈ 636 ms, Approach B ≈ 0.52 ms, nearest neighbour ≈ 0.03 ms** (one laptop-class core). Both are under a second, but A is about 1,200 times slower than B. A's time grows much faster than B's, consistent with $O(P \cdot n^3)$ against a fixed $O(n^3)$: from n = 20 to n = 100 A goes from about 1.1 ms to 636 ms, B from about 0.005 ms to 0.52 ms. Times below about 0.01 ms are at the timer's resolution.



Run time per input (log scale)

9. Trade-offs, limitations and interesting findings

9.1 Trade-offs

- **Quality vs speed.** For realistic batches (up to about 20 orders) A's ability to revise the whole plan pays off: fewest cancellations, close to optimal. B is three orders of magnitude faster and its run time is predictable.
- **Revisable vs committed decisions.** A keeps every customer in the plan and can bring cancelled orders back; B commits each stop. This flexibility is A's strength on small inputs and its cost (many full re-walks) on large ones.
- **Urgency vs geography.** A starts from pure urgency (EDF) and relies on local moves to fix geography. B starts from feasibility and geography (fewest lost, then nearest). When most orders must be dropped, following the map keeps more orders than chasing the most urgent ones.

9.2 Interesting findings

1. **Compare goals in order.** Nearest neighbour often has the best-looking average wait, only because it cancels more customers. A weighted single score would hide this.
2. **Planning first wins for realistic batches.** Up to 20 orders A cancels the fewest at every tightness level and matches the exact optimum's delivered count in 83 to 100% of small inputs.
3. **The ranking flips under overload.** At 50 and 100 orders A cancels the most (63.2 against 54.8 for B and 53.87 for NN at n = 100). Our explanation, not separately tested: EDF sends the rider to far-away urgent ice creams first, and single-stop moves cannot rebuild a route in which most orders must be dropped.
4. **Each approach falls into a trap the other avoids.** B's one-order-at-a-time lookahead misses pairs that cannot both be saved; A's single-move neighbourhood misses fixes that need two moves. This suggests a hybrid.
5. **Cancelling an unreachable order is always safe.** If $t + w(c, v) > d_v$, any detour is at least as long (triangle inequality), so *v* can never arrive on time. This one fact powers B's "doomed" count, the exact solver's bound, and A's cancel-and-stay-put walk.

9.3 Limitations

- Simple world: one rider, all orders known at the start, straight-line travel, no time at the door, no capacity limit.
- Both heuristics are local: A stops when no single move helps; B never takes a choice back.
- No exact optimum for n > 10, so for large *n* we compare methods only against each other.
- Above 33 orders locations repeat, so large-*n* quality results describe overload behaviour, not realistic routing.
- Shelf lives and the 20 km/h speed are assumptions, not measured values.

10. X-factor

- **Exact ground truth with provable pruning.** The brute-force solver is not plain enumeration: it uses a triangle-inequality bound and a dominance rule, and is checked against an unpruned version, so we can report each heuristic's true gap to optimal.
- **An evaluation design that cannot be gamed.** Lexicographic goals, independent re-scoring of every route and the late = 0 check mean no method can look better by delivering late food or dropping customers to shorten waits.
- **Honest, symmetric failure analysis.** For each approach we built a minimal instance where it fails and the other succeeds, and we report where our preferred method (A) loses badly at large *n*.
- **Real-data, fully reproducible pipeline.** 540 inputs from GeoNames, regenerated deterministically from seeds with one command on Windows or macOS/Linux; an interactive demo (`experiments/demo_widget/demo.html`) is verified route-for-route against the Java solvers on 180 inputs.

11. Reproducing the results

Path	Contents
`src/com/delivery/routing/`	Shared model: `Customer`, `DeliveryInstance`, `RouteResult`
`src/com/delivery/tools/`	`Common` (fixed parameters), `GenerateInstance`, `Evaluate`, `NearestNeighbor`, `BruteForce`, `RunExperiments`, worked-example checks
`experiments/approach_A/`	`SolverA.java`, `DemoA.java`, `HOW_IT_WORKS.md`
`experiments/approach_B/`	`SolverB.java`, `DemoB.java`, `HOW_IT_WORKS.md`
`experiments/comparison/`	Run scripts and `plot_results.py`
`experiments/demo_widget/`	Interactive demo of Approach B and its verifier
`data/raw/`, `data/processed/`	Original `MY.txt` and the 540 JSON inputs plus the worked example
`results/`	`raw_results.csv`, `summary.csv`, charts

Requirements: JDK 17+ (no third-party libraries); Python 3.10+ with matplotlib only for charts.

Task	Windows	macOS / Linux
Quick tests	`.\run_tests.bat`	`bash run_tests.sh`
Tests + all 540 inputs + charts	`.\run_tests.bat --all`	`bash run_tests.sh --all`

12. AI usage acknowledgement

Tools used: Claude Code (Anthropic)

How and where it was used: Generate test case. Generate code base on our pseudocode. Generate questions for Q&A.

Parts that are the team's original work: Generate assumptions and goals to optimize. Algorithms and Data structure. The changes to dataset.

Three representative prompts:

1. "Based on MY.txt dataset, generate test cases for our algorithms"
2. "Based on my pseudocode, generate the java code for my algorithms"
3. "Help me generate questions to prepare for QnA"